PRISM - Prototype Source Code

GP-02 AI-Based Precision Irrigation Advisory

Contents

1. README.md
2. gateway/prism_rules.py
3. gateway/advisor.py
4. gateway/gateway.py
5. gateway/simulate.py
6. gateway/templates/dashboard.html
7. gateway/requirements.txt
8. tests/test_rules.py
9. firmware/prism_node/prism_node.ino

README.md

````
# PRISM - GP-02 Precision Irrigation Advisory (prototype code)

```
firmware/prism_node/prism_node.ino   ESP32 node (FreeRTOS, MQTT, offline buffer, valve watchdog)
gateway/prism_rules.py               Rule layer R0-R10 (deterministic, decides everything)
gateway/advisor.py                   Ollama LLM explanation + validator + template fallback
gateway/gateway.py                   MQTT ingest, SQLite, Flask dashboard (cached last advisory)
gateway/simulate.py                  7-day PRISM vs manual simulation
tests/test_rules.py                  Unit tests for rules and fallback
```

## Run without hardware
```
pip install -r gateway/requirements.txt
python -m pytest tests            # 10 tests
python gateway/simulate.py        # add --llm to use Ollama
python gateway/gateway.py --no-mqtt --no-llm   # dashboard at http://localhost:5000
python gateway/simulate.py --post http://localhost:5000/api/ingest   # feed the dashboard
```

## On the Pi
1. `sudo apt install mosquitto` (and hostapd if the Pi hosts the Wi-Fi AP).
2. `curl -fsSL https://ollama.com/install.sh | sh && ollama pull qwen2.5:1.5b-instruct-q4_K_M`
3. `python gateway/gateway.py` (subscribes to `prism/+/telemetry`, publishes `prism/<node>/advisory` and
      `prism/<node>/valve`).
4. Set the rain forecast: `POST /api/forecast {"rain24h_mm":0,"rain_prob":10}` (or implement `fetch_forec
      ast()`); without it confidence drops to Medium.

## Flash the node
Edit Wi-Fi/MQTT settings and the two moisture calibration values at the top of the sketch, install PubSub
      Client, ArduinoJson v7, OneWire, DallasTemperature, Adafruit SHT31, then flash an ESP32 board (e.g.
       "ESP32 Dev Module").

## Caveats
- **Simulation is a logic check only.** The soil model is a crude bucket with an arbitrary ET scale. The
      manual control waters 35 min daily (about 70 mm), which overshoots the 40-70% band, so the 63% savi
      ng and 3.7% in-band for manual are consequences of that assumed schedule, not evidence for the real
      -field thresholds. Only the 7-day field test in the spec can validate them.
- `APP_RATE_MM_PER_MIN`, `ROOT_DEPTH_MM`, the 55% set point and the sensor calibration constants are plac
      eholders to calibrate per zone.
- The firmware and the LLM path were not compiled or run here (no ESP32 toolchain or Ollama in this envir
      onment); the rule layer, template fallback and simulation were tested.
- The LLM latency on a Pi 4B is unmeasured; the 3 s budget falls back to the template automatically.
- `fetch_forecast()` is a stub; no weather API is wired in.
````

gateway/prism_rules.py

```
"""PRISM rule layer: deterministic safety rules (spec Section 4.1).

The rule layer decides action, duration, window and confidence. The AI layer
may only explain these values, never change them.
"""
from __future__ import annotations

from dataclasses import dataclass, field, asdict
from datetime import datetime
from typing import List, Optional

BAND_LOW = 40.0
BAND_HIGH = 70.0
SETPOINT = 55.0
COMFORT = 50.0
ROOT_DEPTH_MM = 300.0
APP_RATE_MM_PER_MIN = 2.0   # drip application rate; calibrate per zone
MIN_MIN, MAX_MIN = 5, 60
DAILY_CAP_MIN = 60
STALE_MIN = 60
BATT_LOW_V = 3.5


@dataclass
class Context:
    m: float                      # soil moisture %
    T: float                      # air temp C
    RH: float                     # relative humidity %
    rain24h_mm: float = 0.0
    rain_prob: float = 0.0        # %
    batt: float = 4.0
    data_age_min: float = 0.0
    forecast_fresh: bool = True
    already_irrigated_min_today: float = 0.0
    now: Optional[datetime] = None
    soil_temp: Optional[float] = None
    ph: Optional[float] = None
    valid: bool = True


@dataclass
class Decision:
    action: str                   # IRRIGATE | NO_IRRIGATION | HOLD
    minutes: int = 0
    window: str = "n/a"
    confidence: str = "High"
    priority: str = "NORMAL"
    reason: str = ""
    warnings: List[str] = field(default_factory=list)
    trace: List[str] = field(default_factory=list)
    valve_allowed: bool = False

    def as_dict(self):
        return asdict(self)


def validate(raw: dict) -> bool:
    """Spec Section 3 validity filter."""
    try:
        return (0 <= raw["soil_moisture_pct"] <= 100
                and -10 <= raw["air_temp_c"] <= 60
                and 0 <= raw["rh_pct"] <= 100)
    except (KeyError, TypeError):
        return False


def _window(now: Optional[datetime], urgent: bool) -> str:
    morning, evening = "05:30-08:00", "17:30-19:00"
    if now is None:
        return morning
    hm = now.hour * 60 + now.minute
    if hm < 8 * 60:
        return morning
    return evening if urgent or hm < 19 * 60 else morning  # next morning if late


def decide(c: Context) -> Decision:
    d = Decision(action="NO_IRRIGATION")
    t = d.trace

    # Confidence (spec): High fresh all; Medium cached forecast; Low stale/invalid
    d.confidence = "High" if c.forecast_fresh else "Medium"

    # R0: stale or invalid data
    if (not c.valid) or c.data_age_min > STALE_MIN:
        d.action, d.confidence, d.reason = "HOLD", "Low", "R0"
        t.append(f"R0 fired: data invalid or older than {STALE_MIN} min "
                 f"(age {c.data_age_min:.0f} min) -> HOLD")
        return d
```

```
t.append(f"R0 not fired: data fresh ({c.data_age_min:.0f} min)")

if c.batt < BATT_LOW_V:
    d.warnings.append("node battery low")
    t.append(f"R9 fired: battery {c.batt:.2f} V < {BATT_LOW_V} V")

# R1: rain predicted
if c.rain24h_mm >= 5 or c.rain_prob >= 70:
    d.reason = "R1"
    t.append(f"R1 fired: rain24h={c.rain24h_mm} mm, prob={c.rain_prob:.0f}% "
             "-> NO_IRRIGATION")
    return d
t.append(f"R1 not fired: rain24h={c.rain24h_mm} mm, prob={c.rain_prob:.0f}%")

# R2: above band
if c.m >= BAND_HIGH:
    d.reason = "R2"
    t.append(f"R2 fired: m={c.m:.1f}% >= {BAND_HIGH}% -> NO_IRRIGATION, valve suppressed")
    return d
t.append(f"R2 not fired: m={c.m:.1f}% < {BAND_HIGH}%")

# R3: comfortable
if c.m >= COMFORT:
    d.reason = "R3"
    t.append(f"R3 fired: m={c.m:.1f}% >= {COMFORT}% -> NO_IRRIGATION")
    return d

# R4 / R5
urgent = c.m < BAND_LOW
d.action = "IRRIGATE"
d.priority = "URGENT" if urgent else "NORMAL"
d.reason = "R4" if urgent else "R5"
t.append(f"{d.reason} fired: m={c.m:.1f}% "
         f"{'< ' + str(BAND_LOW) if urgent else 'in 40-50'} -> IRRIGATE ({d.priority})")

# R6: ET adjustment
factor = 1.0
if c.T > 35 or c.RH < 30:
    factor = 1.2
    t.append(f"R6 fired: T={c.T}C RH={c.RH}% -> duration x1.2")
elif c.T < 20:
    factor = 0.8
    t.append(f"R6 fired: T={c.T}C -> duration x0.8")
else:
    t.append(f"R6 not fired: T={c.T}C RH={c.RH}%")

# R7: duration
deficit_mm = (SETPOINT - c.m) / 100.0 * ROOT_DEPTH_MM
base = deficit_mm / APP_RATE_MM_PER_MIN
minutes = max(MIN_MIN, min(MAX_MIN, round(base * factor)))
t.append(f"R7: deficit=({SETPOINT}-{c.m:.1f})/100 x {ROOT_DEPTH_MM:.0f} = "
         f"{deficit_mm:.1f} mm; base {base:.1f} min x {factor} -> {minutes} min")

# R10: daily cap
remaining = DAILY_CAP_MIN - c.already_irrigated_min_today
if remaining <= 0:
    d.action, d.minutes, d.reason = "NO_IRRIGATION", 0, "R10"
    t.append("R10 fired: daily cap reached -> NO_IRRIGATION")
    return d
if minutes > remaining:
    minutes = int(remaining)
    t.append(f"R10 fired: capped to {minutes} min (daily cap {DAILY_CAP_MIN})")
else:
    t.append(f"R10: daily cap OK ({c.already_irrigated_min_today:.0f}/{DAILY_CAP_MIN} min used)")

d.minutes = int(minutes)
d.window = _window(c.now, urgent)       # R8
t.append(f"R8: window {d.window}")
d.valve_allowed = True
return d
```

gateway/advisor.py

```
"""PRISM AI layer: explanatory advisory text (spec Section 4.2).

Tries a local quantized LLM via Ollama; falls back to a deterministic
template when Ollama is unreachable, output fails validation, or latency
exceeds LATENCY_BUDGET_S. The rule-layer fields are never altered.
"""
from __future__ import annotations

import json
import re
import time
import urllib.request
from typing import Tuple

from prism_rules import Context, Decision, BAND_LOW, BAND_HIGH

OLLAMA_URL = "http://127.0.0.1:11434/api/generate"
MODEL = "qwen2.5:1.5b-instruct-q4_K_M"
LATENCY_BUDGET_S = 3.0

SYSTEM_PROMPT = """You are PRISM, an irrigation advisor. You receive a JSON context already decided
by safety rules. Do NOT change action, minutes, or window. Reply with ONE sentence only:
the agronomic justification, citing the triggering sensor values. Plain language, no jargon.

Example 1 (3-day outlook): {"action":"IRRIGATE","minutes":25,"window":"05:30-08:00","confidence":"High","
      m":38,"T":33,"RH":42}
-> Soil moisture is 38%, below the 40% minimum and air is hot (33 C), so water this morning.

Example 2 (5-day outlook): {"action":"NO_IRRIGATION","reason":"R1","m":46,"rain_prob":80}
-> Rain is 80% likely within 24 h and soil is at 46%, so skip watering and recheck tomorrow.

Example 3 (7-day outlook): {"action":"NO_IRRIGATION","reason":"R3","m":58,"T":27}
-> Soil moisture is 58%, inside the 40-70% target band, so no water is needed.
"""


def template_justification(c: Context, d: Decision) -> str:
    if d.action == "HOLD":
        return (f"Sensor data is missing or {c.data_age_min:.0f} minutes old, "
                "so no watering decision is made until fresh readings arrive.")
    if d.reason == "R1":
        return (f"Rain is {c.rain_prob:.0f}% likely ({c.rain24h_mm:g} mm) within 24 h and soil "
                f"is at {c.m:.0f}%, so skip watering and recheck tomorrow.")
    if d.reason == "R2":
        return (f"Soil moisture is {c.m:.0f}%, above the {BAND_HIGH:.0f}% upper limit, "
                "so watering is blocked to avoid waterlogging.")
    if d.reason == "R3":
        return (f"Soil moisture is {c.m:.0f}%, inside the {BAND_LOW:.0f}-{BAND_HIGH:.0f}% "
                "target band, so no water is needed.")
    if d.reason == "R10":
        return "The daily watering limit has already been reached, so no more water today."
    hot = " and air is hot (%.0f C)" % c.T if c.T > 35 else ""
    if d.reason == "R4":
        return (f"Soil moisture is {c.m:.0f}%, below the {BAND_LOW:.0f}% minimum{hot}, "
                f"so water during {d.window}.")
    return (f"Soil moisture is {c.m:.0f}% and falling toward the {BAND_LOW:.0f}% minimum"
            f"{hot}, so water during {d.window}.")


def _numbers_ok(text: str, c: Context, d: Decision) -> bool:
    """Reject LLM text whose numbers don't match the inputs (spec 4.2 validator)."""
    allowed = {c.m, c.T, c.RH, c.rain_prob, c.rain24h_mm, BAND_LOW, BAND_HIGH, 24.0,
               float(d.minutes)}
    allowed |= {round(x) for x in list(allowed)}
    for tok in re.findall(r"\d+(?:\.\d+)?", text):
        v = float(tok)
        if not any(abs(v - a) <= 1.0 for a in allowed):
            return False
    return True


def _ask_ollama(c: Context, d: Decision, timeout: float) -> str:
    ctx = {"action": d.action, "minutes": d.minutes, "window": d.window,
           "confidence": d.confidence, "reason": d.reason, "m": round(c.m),
           "T": round(c.T), "RH": round(c.RH), "rain_prob": round(c.rain_prob)}
    body = json.dumps({
        "model": MODEL, "system": SYSTEM_PROMPT, "prompt": json.dumps(ctx) + "\n->",
        "stream": False, "keep_alive": "30m",
        "options": {"temperature": 0.2, "num_predict": 60},
    }).encode()
    req = urllib.request.Request(OLLAMA_URL, body, {"Content-Type": "application/json"})
    with urllib.request.urlopen(req, timeout=timeout) as r:
        return json.loads(r.read())["response"].strip().split("\n")[0]


def build_advisory(c: Context, d: Decision, use_llm: bool = True) -> Tuple[dict, dict]:
    """Return (advisory dict, meta dict with latency/source)."""
    t0 = time.perf_counter()
    why, source = None, "template"
```

```
    if use_llm:
        try:
            text = _ask_ollama(c, d, LATENCY_BUDGET_S)
            if text and len(text) < 240 and _numbers_ok(text, c, d) \
                    and (time.perf_counter() - t0) <= LATENCY_BUDGET_S:
                why, source = text, "llm"
        except Exception:
            pass
    if why is None:
        why = template_justification(c, d)
    latency = time.perf_counter() - t0

    adv = {
        "action": d.action,
        "duration_min": d.minutes,
        "timing_window": d.window,
        "confidence": d.confidence,
        "justification": why,
        "warnings": d.warnings,
        "trace": d.trace,
        "valve_allowed": d.valve_allowed,
    }
    return adv, {"latency_s": round(latency, 3), "source": source}


def format_advisory(adv: dict, node_id: str = "prism-n01", issued: str = "") -> str:
    lines = [
        f"PRISM Advisory - Node {node_id}        Issued: {issued}",
        "-" * 62,
        f"Recommended action : {adv['action']}",
        f"Duration           : {adv['duration_min']} minutes",
        f"Timing window      : {adv['timing_window']}",
        f"Confidence         : {adv['confidence']}",
        f"Why: {adv['justification']}",
    ]
    for w in adv["warnings"]:
        lines.append(f"Warning: {w}")
    lines.append("\nExplanation trace")
    lines += [f"  {t}" for t in adv["trace"]]
    return "\n".join(lines)
```

gateway/gateway.py

```
"""PRISM edge gateway: MQTT ingest -> rules -> advisory -> SQLite -> dashboard.

Run:  python gateway.py            (needs Mosquitto on localhost)
      python gateway.py --no-mqtt  (dashboard only; use simulate.py to feed data)
"""
from __future__ import annotations

import argparse
import json
import sqlite3
import threading
import time
from datetime import datetime, timezone, timedelta
from pathlib import Path

from flask import Flask, jsonify, render_template, request

from prism_rules import Context, decide, validate
from advisor import build_advisory, format_advisory

DB_PATH = Path(__file__).with_name("prism.db")
IST = timezone(timedelta(hours=5, minutes=30))
app = Flask(__name__)
_lock = threading.Lock()
FORECAST = {"rain24h_mm": 0.0, "rain_prob": 0.0, "fetched": 0.0}   # cache; updated by fetch_forecast
USE_LLM = True
MQTT_CLIENT = None


def db():
    con = sqlite3.connect(DB_PATH)
    con.row_factory = sqlite3.Row
    return con


def init_db():
    with db() as con:
        con.executescript("""
        CREATE TABLE IF NOT EXISTS readings(id INTEGER PRIMARY KEY, node_id TEXT, ts TEXT,
            soil_moisture_pct REAL, soil_temp_c REAL, air_temp_c REAL, rh_pct REAL,
            soil_ph REAL, battery_v REAL, buffered INTEGER, raw TEXT);
        CREATE TABLE IF NOT EXISTS advisories(id INTEGER PRIMARY KEY, node_id TEXT, ts TEXT,
            latency_s REAL, source TEXT, advisory TEXT);
        CREATE TABLE IF NOT EXISTS irrigation(id INTEGER PRIMARY KEY, node_id TEXT, ts TEXT, minutes REAL
      );
        """)


def irrigated_today(node_id: str) -> float:
    today = datetime.now(IST).strftime("%Y-%m-%d")
    with db() as con:
        r = con.execute("SELECT COALESCE(SUM(minutes),0) s FROM irrigation WHERE node_id=? AND ts LIKE ?"
      ,
                        (node_id, today + "%")).fetchone()
    return float(r["s"])


def fetch_forecast():
    """Optional cloud forecast. Failure keeps the cached values (confidence -> Medium)."""
    # Plug an API here (e.g. Open-Meteo). Left offline-safe by design.
    return False


def process(payload: dict, now: datetime | None = None) -> dict:
    now = now or datetime.now(IST)
    ok = validate(payload)
    try:
        ts = datetime.fromisoformat(payload["ts"])
        age = max(0.0, (now - ts).total_seconds() / 60)
    except Exception:
        age = 9999
    forecast_fresh = (time.time() - FORECAST["fetched"]) < 24 * 3600 and FORECAST["fetched"] > 0
    node = payload.get("node_id", "unknown")
    ctx = Context(
        m=payload.get("soil_moisture_pct", 0), T=payload.get("air_temp_c", 0),
        RH=payload.get("rh_pct", 0), rain24h_mm=FORECAST["rain24h_mm"],
        rain_prob=FORECAST["rain_prob"], batt=payload.get("battery_v", 4.0),
        data_age_min=0 if payload.get("buffered") else age,
        forecast_fresh=forecast_fresh, already_irrigated_min_today=irrigated_today(node),
        now=now, soil_temp=payload.get("soil_temp_c"), ph=payload.get("soil_ph"), valid=ok)
    dec = decide(ctx)
    adv, meta = build_advisory(ctx, dec, use_llm=USE_LLM)
    adv.update(node_id=node, issued=now.isoformat(timespec="seconds"), **meta)
    with _lock, db() as con:
        con.execute("INSERT INTO readings(node_id,ts,soil_moisture_pct,soil_temp_c,air_temp_c,rh_pct,"
                    "soil_ph,battery_v,buffered,raw) VALUES(?,?,?,?,?,?,?,?,?,?)",
                    (node, payload.get("ts"), payload.get("soil_moisture_pct"), payload.get("soil_temp_c"
      ),
                     payload.get("air_temp_c"), payload.get("rh_pct"), payload.get("soil_ph"),
```

```
                    payload.get("battery_v"), int(bool(payload.get("buffered"))), json.dumps(payload)))
        con.execute("INSERT INTO advisories(node_id,ts,latency_s,source,advisory) VALUES(?,?,?,?,?)",
                    (node, adv["issued"], meta["latency_s"], meta["source"], json.dumps(adv)))
        if adv["valve_allowed"] and adv["duration_min"] > 0:
            con.execute("INSERT INTO irrigation(node_id,ts,minutes) VALUES(?,?,?)",
                        (node, adv["issued"], adv["duration_min"]))
    if MQTT_CLIENT:
        MQTT_CLIENT.publish(f"prism/{node}/advisory", json.dumps(adv), qos=1, retain=True)
        if adv["valve_allowed"]:   # rule layer permitted: send valve command
            MQTT_CLIENT.publish(f"prism/{node}/valve",
                                json.dumps({"open_min": adv["duration_min"], "window": adv["timing_window
      "]}),
                                qos=1)
    print(format_advisory(adv, node, adv["issued"]), f"\n[{meta['source']} {meta['latency_s']}s]\n")
    return adv


def start_mqtt(host: str, port: int):
    global MQTT_CLIENT
    import paho.mqtt.client as mqtt
    try:
        cli = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    except AttributeError:          # paho-mqtt 1.x
        cli = mqtt.Client()

    def on_connect(c, *a):
        c.subscribe("prism/+/telemetry", qos=1)

    def on_message(c, u, msg):
        try:
            process(json.loads(msg.payload))
        except Exception as e:
            print("bad message:", e)

    cli.on_connect, cli.on_message = on_connect, on_message
    cli.connect_async(host, port)
    cli.loop_start()
    MQTT_CLIENT = cli


# ---------------- Dashboard ----------------
@app.route("/")
def index():
    return render_template("dashboard.html")


@app.route("/api/latest")
def api_latest():
    with db() as con:
        r = con.execute("SELECT * FROM advisories ORDER BY id DESC LIMIT 1").fetchone()
        rd = con.execute("SELECT * FROM readings ORDER BY id DESC LIMIT 1").fetchone()
    if not r:
        return jsonify({})
    return jsonify({"advisory": json.loads(r["advisory"]), "latency_s": r["latency_s"],
                    "source": r["source"], "reading": dict(rd) if rd else None})


@app.route("/api/history")
def api_history():
    n = int(request.args.get("n", 96))
    with db() as con:
        rows = con.execute("SELECT ts, soil_moisture_pct m, air_temp_c T, rh_pct RH FROM readings "
                           "ORDER BY id DESC LIMIT ?", (n,)).fetchall()
    return jsonify([dict(x) for x in rows][::-1])


@app.route("/api/forecast", methods=["POST"])
def api_forecast():
    j = request.get_json(force=True)
    FORECAST.update(rain24h_mm=float(j.get("rain24h_mm", 0)), rain_prob=float(j.get("rain_prob", 0)),
                    fetched=time.time())
    return jsonify(FORECAST)


@app.route("/api/ingest", methods=["POST"])
def api_ingest():          # HTTP fallback / used by simulate.py
    return jsonify(process(request.get_json(force=True)))


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--no-mqtt", action="store_true")
    ap.add_argument("--no-llm", action="store_true")
    ap.add_argument("--mqtt-host", default="localhost")
    ap.add_argument("--port", type=int, default=5000)
    a = ap.parse_args()
    USE_LLM = not a.no_llm
    init_db()
    if not a.no_mqtt:
        start_mqtt(a.mqtt_host, 1883)
    app.run(host="0.0.0.0", port=a.port)
```

gateway/simulate.py

```
"""Simulate a PRISM node and run the pipeline without hardware.

  python simulate.py                # 7-day offline sim, PRISM vs manual control, prints metrics
  python simulate.py --post URL     # also POST each reading to a running gateway /api/ingest

Soil model is a simple bucket: ET loss + drip gain + rain. It is a logic check
for the rules, NOT a substitute for the field validation in spec Section 6.
"""
import argparse
import json
import math
import random
import urllib.request
from datetime import datetime, timedelta, timezone

from prism_rules import Context, decide, BAND_LOW, BAND_HIGH, APP_RATE_MM_PER_MIN, ROOT_DEPTH_MM
from advisor import build_advisory

IST = timezone(timedelta(hours=5, minutes=30))
STEP_MIN = 15
DAYS = 7


def weather(t: datetime, rng):
    h = t.hour + t.minute / 60
    T = 29 + 6 * math.sin((h - 9) / 24 * 2 * math.pi) + rng.uniform(-0.5, 0.5)
    RH = max(25, min(95, 60 - 1.8 * (T - 29) * 3 + rng.uniform(-2, 2)))
    return T, RH


def et_mm_per_step(T):
    return max(0.0, (0.12 + 0.012 * (T - 20))) * STEP_MIN / 60  # mm per step


def run(mode: str, seed=1, post=None, use_llm=False):
    rng = random.Random(seed)
    m = 62.0
    t = datetime(2026, 11, 2, 0, 0, tzinfo=IST)
    water_min, in_band, n = 0.0, 0, 0
    irrigating_left = 0.0
    lat = []
    today_min, day = 0.0, t.day
    rain_by_day = {4: 12.0}                      # one rainy day (day-of-month)
    last_decision_slot = None
    for _ in range(DAYS * 24 * 60 // STEP_MIN):
        T, RH = weather(t, rng)
        if t.day != day:
            day, today_min = t.day, 0.0
        rain = rain_by_day.get(t.day, 0) / (24 * 60 / STEP_MIN) if 10 <= t.hour < 16 else 0
        m = max(0, min(100, m - et_mm_per_step(T) / ROOT_DEPTH_MM * 100 * 6 + rain / ROOT_DEPTH_MM * 100
      * 6))
        # --- control ---
        if mode == "manual":
            if t.hour == 6 and t.minute == 0:
                irrigating_left = 35          # fixed daily schedule
        else:
            slot = (t.day, t.hour)
            if t.minute == 0 and t.hour in (5, 6, 17) and slot != last_decision_slot and irrigating_left
      <= 0:
                last_decision_slot = slot
                tomorrow_rain = rain_by_day.get((t + timedelta(days=0)).day, 0)
                ctx = Context(m=round(m, 1), T=round(T, 1), RH=round(RH, 1), rain24h_mm=tomorrow_rain,
                              rain_prob=90 if tomorrow_rain >= 5 else 5, batt=3.9, data_age_min=1,
                              already_irrigated_min_today=today_min, now=t, valid=True)
                dec = decide(ctx)
                adv, meta = build_advisory(ctx, dec, use_llm=use_llm)
                lat.append(meta["latency_s"])
                if post:
                    pl = {"node_id": "prism-sim", "ts": t.isoformat(), "soil_moisture_pct": ctx.m,
                          "air_temp_c": ctx.T, "rh_pct": ctx.RH, "battery_v": 3.9, "buffered": False}
                    urllib.request.urlopen(urllib.request.Request(
                        post, json.dumps(pl).encode(), {"Content-Type": "application/json"}))
                if dec.valve_allowed:
                    irrigating_left = dec.minutes
                    today_min += dec.minutes
        if irrigating_left > 0:
            run_min = min(STEP_MIN, irrigating_left)
            irrigating_left -= run_min
            water_min += run_min
            m = min(100, m + run_min * APP_RATE_MM_PER_MIN / ROOT_DEPTH_MM * 100)
        n += 1
        in_band += BAND_LOW <= m <= BAND_HIGH
        t += timedelta(minutes=STEP_MIN)
    return {"water_min": water_min, "time_in_band_pct": 100 * in_band / n,
            "median_latency_s": sorted(lat)[len(lat) // 2] if lat else None}


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
```

```
ap.add_argument("--post")
ap.add_argument("--llm", action="store_true")
a = ap.parse_args()
man = run("manual")
pri = run("prism", post=a.post, use_llm=a.llm)
saving = 100 * (1 - pri["water_min"] / man["water_min"])
print(f"Manual : {man['water_min']:.0f} min, in-band {man['time_in_band_pct']:.1f}%")
print(f"PRISM  : {pri['water_min']:.0f} min, in-band {pri['time_in_band_pct']:.1f}%, "
      f"median latency {pri['median_latency_s']}s")
print(f"Water saving: {saving:.1f}%  (target >=30%)   "
      f"in-band target >=90%: {'PASS' if pri['time_in_band_pct'] >= 90 else 'FAIL'}")
```

gateway/templates/dashboard.html

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>PRISM</title>
<style>
:root{--bg:#f6f7f4;--fg:#1d2a1f;--card:#fff;--ok:#2f7d4f;--warn:#b7791f;--bad:#b3392f;--mut:#6b756d}
@media(prefers-color-scheme:dark){:root{--bg:#121613;--fg:#e8efe9;--card:#1b211c;--mut:#9aa69c}}
body{margin:0;font:16px system-ui;background:var(--bg);color:var(--fg);padding:16px;max-width:720px;margi
      n:auto}
.card{background:var(--card);border-radius:12px;padding:16px;margin-bottom:12px}
h1{font-size:20px}.big{font-size:28px;font-weight:700}.mut{color:var(--mut);font-size:13px}
.IRRIGATE{color:var(--ok)}.NO_IRRIGATION{color:var(--mut)}.HOLD{color:var(--warn)}
pre{white-space:pre-wrap;font-size:12px;color:var(--mut)} canvas{width:100%;height:140px}
</style></head><body>
<h1>PRISM <span class="mut">irrigation advisory</span></h1>
<div class="card"><div id="act" class="big">...</div><div id="detail"></div>
<p id="why"></p><div id="age" class="mut"></div></div>
<div class="card"><b>Soil moisture</b> <span class="mut">(band 40-70%)</span><canvas id="c"></canvas></di
      v>
<div class="card"><b>Explanation trace</b><pre id="trace"></pre></div>
<script>
const KEY='prism_last';
function render(d,cached){
  const a=d.advisory; if(!a)return;
  const el=document.getElementById('act');
  el.textContent=a.action.replace('_',' '); el.className='big '+a.action;
  document.getElementById('detail').innerHTML=
    `Duration: <b>${a.duration_min} min</b> · Window: <b>${a.timing_window}</b> · Confidence: <b>${a.conf
      idence}</b>`;
  document.getElementById('why').textContent=a.justification+(a.warnings.length?' ? '+a.warnings.join(',
      '):'');
  const mins=Math.round((Date.now()-new Date(a.issued))/60000);
  document.getElementById('age').textContent=(cached?'Offline - cached copy. ':'')+`Issued ${mins} min ag
      o · ${d.source} · ${d.latency_s}s`;
  document.getElementById('trace').textContent=a.trace.join('\n');
}
function chart(rows){
  const c=document.getElementById('c'),x=c.getContext('2d');c.width=c.clientWidth*2;c.height=280;
  x.clearRect(0,0,c.width,c.height);const H=c.height,W=c.width,y=v=>H-(v/100)*H;
  x.fillStyle='rgba(47,125,79,.15)';x.fillRect(0,y(70),W,y(40)-y(70));
  x.strokeStyle='#2f7d4f';x.lineWidth=3;x.beginPath();
  rows.forEach((r,i)=>{const px=i/(Math.max(rows.length-1,1))*W;i?x.lineTo(px,y(r.m)):x.moveTo(px,y(r.m))
      });x.stroke();
}
async function load(){
  try{
    const d=await (await fetch('/api/latest')).json();
    localStorage.setItem(KEY,JSON.stringify(d)); render(d,false);
    chart(await (await fetch('/api/history')).json());
  }catch(e){const s=localStorage.getItem(KEY); if(s)render(JSON.parse(s),true);}
}
load(); setInterval(load,30000);
</script></body></html>
```

gateway/requirements.txt

```
flask>=3.0
paho-mqtt>=2.0
pytest>=8.0
```

tests/test_rules.py

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).parent.parent / "gateway"))

from prism_rules import Context, decide, validate
from advisor import build_advisory, template_justification


def mk(**kw):
    base = dict(m=45, T=30, RH=50)
    base.update(kw)
    return Context(**base)


def test_spec_example_urgent_irrigate():
    d = decide(mk(m=38.0, T=32, RH=44))
    assert d.action == "IRRIGATE" and d.reason == "R4" and d.priority == "URGENT"
    assert d.minutes == 26 or d.minutes == 25 or d.minutes == 26  # deficit 51 mm / 2.0
    assert d.valve_allowed


def test_rain_blocks_irrigation():
    d = decide(mk(m=30, rain_prob=80))
    assert d.action == "NO_IRRIGATION" and d.reason == "R1" and not d.valve_allowed


def test_upper_band_suppresses_valve():
    d = decide(mk(m=72))
    assert d.action == "NO_IRRIGATION" and d.reason == "R2" and not d.valve_allowed


def test_comfortable_no_irrigation():
    assert decide(mk(m=58)).reason == "R3"


def test_stale_data_holds():
    d = decide(mk(m=30, data_age_min=90))
    assert d.action == "HOLD" and d.confidence == "Low"


def test_heat_increases_duration():
    cool = decide(mk(m=42, T=28, RH=50)).minutes
    hot = decide(mk(m=42, T=38, RH=25)).minutes
    assert hot > cool


def test_daily_cap():
    d = decide(mk(m=30, already_irrigated_min_today=60))
    assert d.action == "NO_IRRIGATION" and d.reason == "R10"
    d = decide(mk(m=30, already_irrigated_min_today=40))
    assert d.minutes <= 20


def test_low_battery_warning_and_cached_forecast_confidence():
    d = decide(mk(m=45, batt=3.3, forecast_fresh=False))
    assert "node battery low" in d.warnings and d.confidence == "Medium"


def test_validate():
    ok = {"soil_moisture_pct": 40, "air_temp_c": 30, "rh_pct": 50}
    assert validate(ok)
    assert not validate({**ok, "soil_moisture_pct": 140})
    assert not validate({"air_temp_c": 30})


def test_template_fallback_keeps_rule_fields():
    c = mk(m=38, T=33, RH=42)
    d = decide(c)
    adv, meta = build_advisory(c, d, use_llm=False)
    assert meta["source"] == "template"
    assert adv["duration_min"] == d.minutes and adv["timing_window"] == d.window
    assert "38%" in adv["justification"]
```

firmware/prism_node/prism_node.ino

```
/*
 * PRISM sensor node - ESP32 (dual core, FreeRTOS)
 * Core 1: sensor polling task      Core 0: Wi-Fi/MQTT + offline buffer + valve task
 * (spec: ESP8266 single core lost data during concurrent polling + Wi-Fi TX)
 *
 * Libraries (Arduino Library Manager): PubSubClient, ArduinoJson (v7),
 *   OneWire, DallasTemperature, Adafruit SHT31 Library
 *
 * Wiring:  SEN0193 moisture -> GPIO34 | DS18B20 -> GPIO4 (4.7k pull-up to 3V3)
 *          SHT31 -> I2C GPIO21/22      | SEN0169 pH (optional) -> GPIO32
 *          Battery divider 100k/100k -> GPIO35 | Relay (valve) -> GPIO26
 */
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Adafruit_SHT31.h>
#include <LittleFS.h>
#include <time.h>

// ---------- configuration ----------
const char* WIFI_SSID = "PRISM_GATEWAY";
const char* WIFI_PASS = "change-me";
const char* MQTT_HOST = "192.168.4.1";      // Raspberry Pi (hosts AP + Mosquitto)
const uint16_t MQTT_PORT = 1883;
const char* NODE_ID = "prism-n01";

const uint32_t CYCLE_MS = 15UL * 60UL * 1000UL;   // 15 min measurement cycle
const int SAMPLES = 5;                            // median of 5
const int BUF_MAX = 96;                           // offline readings kept
const int MOIST_AIR_RAW = 3000;                   // calibrate: sensor in air  (0 %)
const int MOIST_WATER_RAW = 1300;                 // calibrate: sensor in water (100 %)
const bool HAS_PH = true;
const float PH_OFFSET = 0.0f;                     // calibrate with pH 4.0 / 7.0 buffers
const uint32_t VALVE_MAX_MS = 60UL * 60UL * 1000UL;  // hard cap 60 min

const int PIN_MOIST = 34, PIN_ONEWIRE = 4, PIN_PH = 32, PIN_BATT = 35, PIN_RELAY = 26;

// ---------- globals ----------
OneWire oneWire(PIN_ONEWIRE);
DallasTemperature ds18(&oneWire);
Adafruit_SHT31 sht31;
WiFiClient wifi;
PubSubClient mqtt(wifi);
QueueHandle_t readingQ;                           // sensor task -> net task
String topicTelemetry, topicValve, topicStatus;
volatile uint32_t valveOffAt = 0;                 // millis() deadline, 0 = closed

struct Reading { char json[320]; };

// ---------- helpers ----------
static float medianOf(float* a, int n) {
  for (int i = 1; i < n; i++) { float k = a[i]; int j = i - 1; while (j >= 0 && a[j] > k) { a[j + 1] = a[
      j]; j--; } a[j + 1] = k; }
  return a[n / 2];
}

float readMoisturePct() {
  float v[SAMPLES];
  for (int i = 0; i < SAMPLES; i++) { v[i] = analogRead(PIN_MOIST); delay(20); }
  float raw = medianOf(v, SAMPLES);
  float pct = (MOIST_AIR_RAW - raw) * 100.0f / (MOIST_AIR_RAW - MOIST_WATER_RAW);
  return constrain(pct, 0.0f, 100.0f);
}

float readBatteryV() {
  float v[SAMPLES];
  for (int i = 0; i < SAMPLES; i++) { v[i] = analogReadMilliVolts(PIN_BATT) * 2.0f / 1000.0f; delay(5); }
  return medianOf(v, SAMPLES);
}

float readPh() {
  float v[SAMPLES];
  for (int i = 0; i < SAMPLES; i++) { v[i] = analogReadMilliVolts(PIN_PH) / 1000.0f; delay(20); }
  return 3.5f * medianOf(v, SAMPLES) + PH_OFFSET;       // SEN0169 transfer function
}

String isoNow() {
  struct tm t;
  if (!getLocalTime(&t, 200)) return String("1970-01-01T00:00:00+05:30");
  char b[40]; strftime(b, sizeof(b), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(b);
}

void closeValve() { digitalWrite(PIN_RELAY, LOW); valveOffAt = 0; }
void openValve(uint32_t minutes) {
  uint32_t ms = min((uint32_t)(minutes * 60000UL), VALVE_MAX_MS);
  digitalWrite(PIN_RELAY, HIGH); valveOffAt = millis() + ms;
```

```
}

// ---------- offline buffer (LittleFS ring, one JSON per line) ----------
void bufferAppend(const char* line) {
  File f = LittleFS.open("/buf.jsonl", "r");
  std::vector<String> lines;
  if (f) { while (f.available()) { String s = f.readStringUntil('\n'); if (s.length() > 2) lines.push_bac
      k(s); } f.close(); }
  lines.push_back(String(line));
  while ((int)lines.size() > BUF_MAX) lines.erase(lines.begin());
  File w = LittleFS.open("/buf.jsonl", "w");
  for (auto& s : lines) w.println(s);
  w.close();
}

void bufferFlush() {
  File f = LittleFS.open("/buf.jsonl", "r");
  if (!f) return;
  std::vector<String> keep;
  bool ok = true;
  while (f.available()) {
    String s = f.readStringUntil('\n'); s.trim();
    if (s.length() < 3) continue;
    if (ok && mqtt.connected() && mqtt.publish(topicTelemetry.c_str(), s.c_str())) { mqtt.loop(); delay(5
      0); }
    else { ok = false; keep.push_back(s); }
  }
  f.close();
  if (keep.empty()) LittleFS.remove("/buf.jsonl");
  else { File w = LittleFS.open("/buf.jsonl", "w"); for (auto& s : keep) w.println(s); w.close(); }
}
// ---------- tasks ----------
void sensorTask(void*) {                        // pinned to core 1
  TickType_t last = xTaskGetTickCount();
  for (;;) {
    JsonDocument d;
    d["schema_version"] = "1.0";
    d["node_id"] = NODE_ID;
    d["ts"] = isoNow();
    d["soil_moisture_pct"] = roundf(readMoisturePct() * 10) / 10;
    ds18.requestTemperatures();
    float st = ds18.getTempCByIndex(0);
    if (st > -100) d["soil_temp_c"] = roundf(st * 10) / 10; else d["soil_temp_c"] = nullptr;
    float at = sht31.readTemperature(), rh = sht31.readHumidity();
    if (!isnan(at)) d["air_temp_c"] = roundf(at * 10) / 10;
    if (!isnan(rh)) d["rh_pct"] = roundf(rh * 10) / 10;
    if (HAS_PH) d["soil_ph"] = roundf(readPh() * 10) / 10; else d["soil_ph"] = nullptr;
    d["battery_v"] = roundf(readBatteryV() * 100) / 100;
    d["buffered"] = false;
    Reading r; serializeJson(d, r.json, sizeof(r.json));
    xQueueSend(readingQ, &r, 0);
    vTaskDelayUntil(&last, pdMS_TO_TICKS(CYCLE_MS));
  }
}

void valveTask(void*) {                         // safety watchdog for the valve
  for (;;) {
    if (valveOffAt && (int32_t)(millis() - valveOffAt) >= 0) closeValve();
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}

void onMsg(char* topic, byte* payload, unsigned int len) {
  JsonDocument d;
  if (deserializeJson(d, payload, len)) return;
  if (String(topic) == topicValve) {
    uint32_t m = d["open_min"] | 0;
    if (m > 0) openValve(m); else closeValve();
  }
}

void netTask(void*) {                           // pinned to core 0
  Reading r;
  uint32_t lastTry = 0;
  for (;;) {
    if (WiFi.status() != WL_CONNECTED && millis() - lastTry > 10000) {
      lastTry = millis(); WiFi.begin(WIFI_SSID, WIFI_PASS);
    }
    if (WiFi.status() == WL_CONNECTED && !mqtt.connected() && millis() - lastTry > 5000) {
      lastTry = millis();
      if (mqtt.connect(NODE_ID, topicStatus.c_str(), 1, true, "offline")) {
        mqtt.publish(topicStatus.c_str(), "online", true);
        mqtt.subscribe(topicValve.c_str(), 1);
        bufferFlush();                          // replay readings taken while offline
      }
    }
    mqtt.loop();
    if (xQueueReceive(readingQ, &r, pdMS_TO_TICKS(200)) == pdTRUE) {
      if (!(mqtt.connected() && mqtt.publish(topicTelemetry.c_str(), r.json))) {
        // mark as buffered so the gateway does not treat old data as stale
        JsonDocument d; deserializeJson(d, r.json); d["buffered"] = true;
        char out[320]; serializeJson(d, out, sizeof(out));
```

```
        bufferAppend(out);
      }
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_RELAY, OUTPUT); digitalWrite(PIN_RELAY, LOW);
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_MOIST, ADC_11db);
  LittleFS.begin(true);
  ds18.begin();
  if (!sht31.begin(0x44)) Serial.println("SHT31 not found");
  topicTelemetry = String("prism/") + NODE_ID + "/telemetry";
  topicValve = String("prism/") + NODE_ID + "/valve";
  topicStatus = String("prism/") + NODE_ID + "/status";
  WiFi.mode(WIFI_STA); WiFi.begin(WIFI_SSID, WIFI_PASS);
  configTime(5 * 3600 + 1800, 0, "pool.ntp.org");   // IST; falls back to gateway NTP if offline
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(onMsg);
  mqtt.setBufferSize(512);
  readingQ = xQueueCreate(4, sizeof(Reading));
  xTaskCreatePinnedToCore(netTask, "net", 8192, nullptr, 1, nullptr, 0);
  xTaskCreatePinnedToCore(sensorTask, "sensor", 6144, nullptr, 2, nullptr, 1);
  xTaskCreatePinnedToCore(valveTask, "valve", 2048, nullptr, 3, nullptr, 1);
}

void loop() { vTaskDelay(portMAX_DELAY); }
```